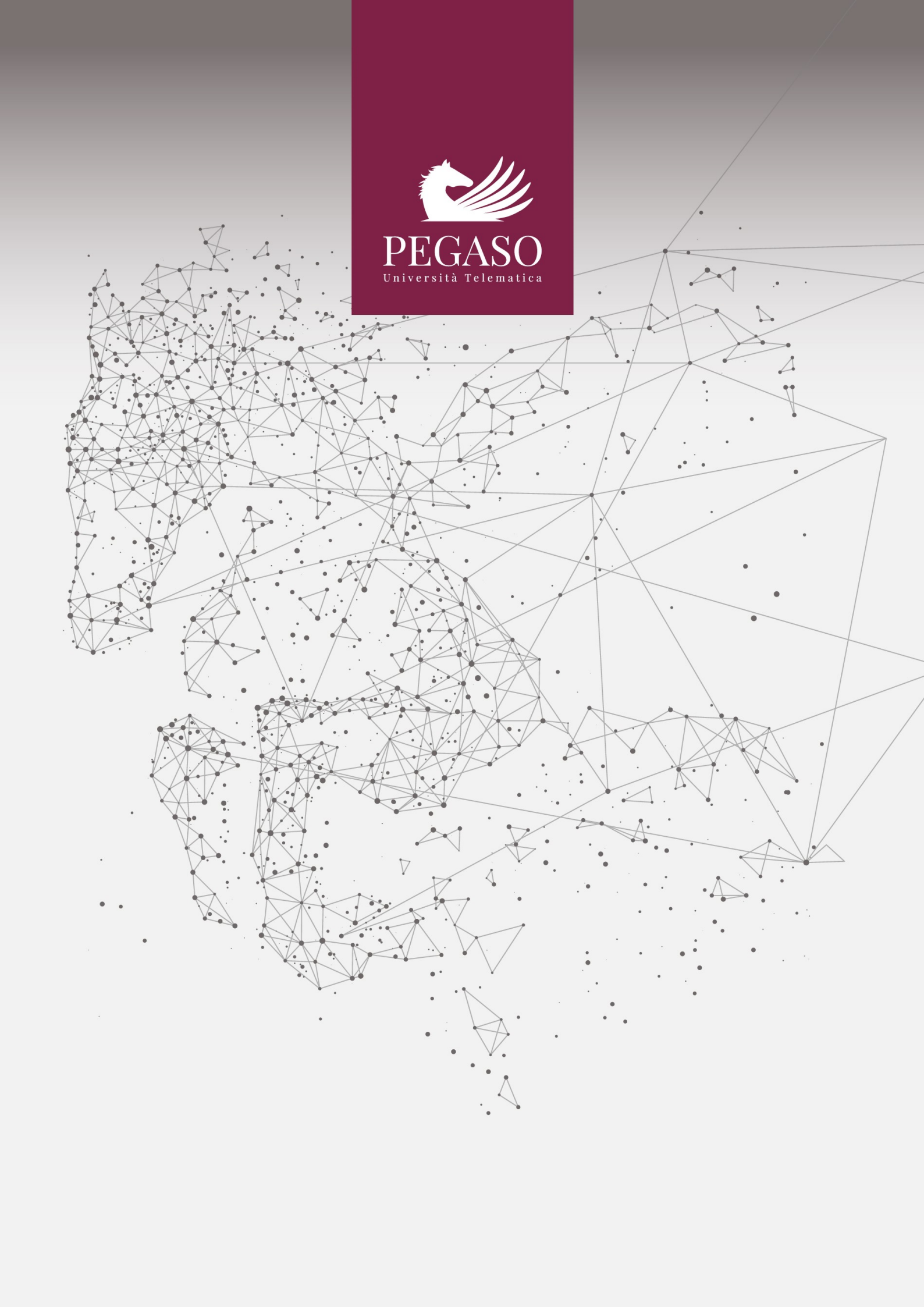

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. GESTIONE DEL FLUSSO DI ESECUZIONE 3
2. COSTRUTTO IF 4
3. L'OPERATORE TERNARIO 8
4. COSTRUTTO SWITCH 10
5. ERRORI TIPICI NELL'UTILIZZO DI IF E SWITCH 15
6. COSTRUTTI WHILE E FOR 16
7. COSTRUTTI DO-WHILE E FOREACH 21
BIBLIOGRAFIA 23

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Gestione del flusso di esecuzione

In ogni linguaggio di programmazione esistono costrutti che permettono all'utente di controllare e gestire la sequenza di esecuzione delle istruzioni a runtime. Essenzialmente possiamo dividere questi costrutti in due categorie principali:

- Condizioni (o strutture di controllo decisionali): permettono di effettuare una scelta tra l'esecuzione di istruzioni diverse a seconda che sia verificata una specificata condizione, la quale coincide con i risultato di un'operazione booleana
    - if, switch, operatore ternario
- Cicli (strutture di controllo iterative): permettono di decidere il numero di esecuzioni di determinate istruzioni.
    - while, for, do-while, foreach (ciclo for migliorato)

  Tali costrutti possono essere annidati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Costrutto if

Quando in un programma spesso c'e' bisogno di scegliere tra operazioni diverse, si usa l'istruzione condizionale if-else che consente di prendere semplici decisioni basate su valori immagazzinati. In fase di runtime la JVM testa un'espressione booleana e, a seconda che essa risulti vera o falsa, esegue un certo blocco di istruzioni oppure no. Un'espressione booleana è un'espressione che come risultato può restituire solo valori di tipo boolean, vale a dire true o false.

La sintassi generale è la seguente:

```
if (espressione booleana) {
istruzione_1;
….
istruzione_k;
} else {
istruzione_k+1;
…
istruzione_n;
}
```

Se l'espressione booleana si verifica (ovvero vale true) si esegue il blocco di istruzioni da 1 a k (potrebbe anche essere una sola istruzione), se non si verifica (ovvero vale false) allora si esegue il blocco di istruzione da k+1 a n (anche in questo caso potrebbe anche essere una sola istruzione). Il ramo else potrebbe anche non esserci e quindi indicare di eseguire le istruzioni da 1 a k in caso di true e basta.

I due comandi alternativi sono detti anche rami con else che può essere opzionale come già detto.

Qui abbiamo indicato il caso generico di più istruzioni per ogni ramo che sono racchiusi fra parentesi graffe {...} denominati blocchi di codice. Ovviamente potrebbe anche essere una sola istruzione. In tal caso le parentesi graffe potrebbero anche non essere messe, ma si consiglia comunque di usarle anche per circondare un'unica istruzione. Infatti, questa pratica aggiunge leggibilità e favorisce l'evoluzione del codice.

Nell'esempio in alto di Fig. 1 l'istruzione di stampa viene eseguita se e solo se la variabile numeroLati ha valore 3. In quel caso l'espressione booleana (numeroLati==3) vale true e quindi sarebbe eseguita l'istruzione che segue l'espressione. Se invece l'espressione risultasse false, sarebbe eseguita direttamente la prima eventuale istruzione che segue l'istruzione di stampa. Possiamo estendere la

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

potenzialità del costrutto if mediante parola chiave else come mostrato nell'esempio in basso. La seconda stampa viene eseguita se l'espressione risulta false.

```
…
if (numeroLati == 3)
       System.out.println ("Questo è un triangolo");
// il flusso di esecuzione riprende da qui in ogni caso
System.out.println («Qual è la prossima figura geometrica?");

…
if (numeroLati == 3)
       System.out.println ("Questo è un triangolo");
else
       System.out.println ("Questo non è un triangolo");
…
```

Fig. 1: Esempi di costrutti if

Un'espressione booleana è un'espressione che come risultato può restituire solo valori di tipo boolean, vale a dire true o false. Essa si avvale di operatori di confronto e, se necessario, di operatori logici. Sono espressioni booleane le seguenti:

- x==10 (la variabile x `e uguale a 10)
- x!=10 (la variabile x `e diversa da 10)
- (x/2)<=10 (l'espressione x/2 `e minore o uguale a 10)
- (x/2)>=(2+y) (l'espressione x/2 `e maggiore o uguale all'espressione. 2+y)
- (trovato==true) (la variabile boolean è verificata)
- ((x/2)<=10) || ((z/3)>=(2+y)) (verificata se una delle due espressioni risulta vera)

I comandi if-else possono essere annidati. In Fig. 2 sono mostrati due esempi diversi. Come si può vedere in generale non servono le parentesi perché if-else è un unico commando. Nel farlo però il codice diventa sempre più complesso generando situazioni equivoche e bisogna fare attenzione.

```
…
if (saldo >= 0)
       if (saldo > 0)
              System.out.println (" Saldo positivo !");
       else
              System.out.println (" Saldo zero !");
…
if (saldo >=0)  {
       if (saldo >0)
              System.out.println (" Saldo positivo !");
} else
       System . out . println (" Saldo zero o negativo");
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Fig. 2: Esempi di costrutti if annidati

Il primo frammento di codice mostra un if che annida un costrutto if-else (else fa riferimento al secondo if), il secondo mostra un costrutto if-else che annida un costrutto if (else fa riferimento al primo if). In questo secondo caso sono necessarie le parentesi per far riferire l'else al primo if, altrimenti si riferirebbe al secondo.

ATTENZIONE: L'uso attento di spazi e "a capo" nel programma (indentazione) semplifica la lettura e la comprensione del codice.

Si consiglia sempre di usare un blocco di codice (mediante l'uso di parentesi graffe) per circondare anche un'unica istruzione. Questa pratica oltre ad aggiungere leggibilità consente anche di aggiungere istruzioni in un secondo momento, come spesso succede.

a. if concetenati

Un caso particolare di if-else annidati sono gli if-else concatenati: il ramo else è costituito da un altro if . L'espressione booleana del secondo if considera un caso alternativo a quello considerato dalla condizione del primo if (o degli if precedenti). Da notare l'indentazione: di solito si scrive else if nella stessa riga.

Il funzionamento è abbastanza intuitivo. Se espressione-booleana1 fosse true verrebbero eseguite le isruzione del primo blocco di codice (istruzione_1;….istruzione_k;) e non sarebbero eseguite le istruzioni presenti negli altri blocchi. Infatti, non venendo eseguito il primo blocco dopo il primo else non sono eseguiti neanche gli if che ci sono dopo. Quindi le altre condizioni booleane saranno ignorate e il flusso di esecuzione del codice si sposterà fuori dall'intero costrutto (dopo l'ultima parentesi graffa che definisce il blocco di codice della clausola else).

Se invece espressione-booleana1 fosse false allora si valuterebbe espressione-booleana2 e nel caso fosse true si eseguirebbero le relative istruzioni (istruzione_k+1; …istruzione_j;) ignorando le altre linee di codice. Continuando la valutazione, se anche espressione-booleana2 fosse false si passerebbe a valutare espressione-booleana3 e se valesse true si eseguirebbero le relative istruzioni (istruzione_j+1;…istruzione_h) ignorando il resto del codice. Infine se tutte le espressioni booleane risultassero non verificate (ovvero valessero false) allora verrebbero eseguiti tutte le istruzioni contenuti nel blocco di codice relativo alla clausola else (istruzione_k+1; … istruzione_n). In Fig. 3 un if concatenato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
…
if (numeroLati == 3)
       System.out.println ("Questo è un triangolo");
else if (numeroLati == 4)
       System.out.println ("Questo è un quadrilatero");
else if (numeroLati == 5)
       System.out.println ("Questo è un pentagono");
else if (numeroLati == 6)
       System.out.println ("Questo è un esagono");
else
       System.out.println ("Troppi lati! ");
…
```

Fig. 3: Esempio di if concatenati

.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. L'operatore ternario

Esiste un operatore che talvolta può sostituire un costrutto if. Si tratta del cosiddetto operatore ternario (detto anche operatore condizionale) che può regolare il flusso di esecuzione come una condizione. La sintassi è la seguente:

var = (espressione booleana)? valore1 : valore2;

dove:

- var: variabile a cui viene assegnato il valore
- espressione booleana: è condizione di test che viene valutata e restituisce un valore booleano, ovvero vero o falso
- valore1: se espressione booleana viene valutato come 'true', allora value1 viene assegnato a var
- valore2:se espressione booleana viene valutato come 'false', value2 viene assegnato a var

Questo operatore è chiamato come operatore ternario poiché l'operatore ternario utilizza 3 operandi, il primo è un'espressione booleana che restituisce vero o falso, il secondo è il risultato quando l'espressione booleana restituisce vero e il terzo è il risultato quando l'espressione booleana restituisce falso. Inoltre come tutti gli operatori ritorna un valore.

Un requisito indispensabile è che il tipo della variabile e quello restituito da espr1 e espr2 siano compatibili. E' escluso il tipo void.

Inoltre il valore ritornato deve per forza essere catturato:

- con una variabile
- passando il suo risultato ad un metodo come parametro di input
- restituendo il suo valore in output tramite un comando return di un metodo
- in un'espressione switch

Non è possibile ritornare un valore senza usarlo pena un messaggio di errore di compilazione.

L'operatore ternario non può essere considerato un sostituto del costrutto if, ma è molto comodo in alcune situazioni.

```
…
String resultString = (5> 1)? 'PASS': 'FAIL';

String resultValue = (x >= y)? 'X è maggiore o forse uguale a
y': 'x è minore di y';
```

Fig. 4: Esempio di operatore ternario

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In Fib. 4 un esempio: l'operatore ternario valuta la condizione di test (5> 1), se restituisce true quindi assegna il valore1, ovvero 'PASS' e assegna 'FAIL' se restituisce false. Poiché (5> 1) è vero, resultString il valore viene assegnato come 'PASS'.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Costrutto switch

In alcuni casi un comando if può diventare un po' lungo da scrivere.

Consideriamo il frammento di codice in Fig. 5 che sulla base del valore della variabile giorno (definita come intero) stampa la stringa del giorno della settimana corrispondente. Esempio: un programma che legge un numero e lo trasforma in un giorno.

```
…
if (giorno ==1) System.out.println (" Lunedi ");
else if ( giorno == 2) System . out . println (" Martedi ");
else if ( giorno == 3) System . out . println (" Mercoledi ");
else if ( giorno == 4) System . out . println (" Giovedi ");
else if ( giorno == 5) System . out . println (" Venerdi ");
else if ( giorno == 6) System . out . println (" Sabato ");
else if ( giorno == 7) System . out . println (" Domenica ");
else System.out.println (" Numero errato ");
…
```

Fig. 5: Esempio di if concatenati con giorni della settimana

In questi casi, di if concatenati andando a confrontare il risultato di una espressione di tipo int o char con un numero di letterali alternativi (ad es. 1,2,3,...) l'if può essere sostituito da uno switch che consente di eseguire determinate istruzioni piuttosto che altre in base al valore che sarà passato durante l'esecuzione al costrutto.

Il costrutto switch si presenta quindi in alcune situazioni come alternativo al costrutto if: Consente di eseguire determinate istruzioni piuttosto che altre, in base al valore passato durante l'esecuzione del costrutto.

Di seguito presentiamo la sintassi:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
switch (variabile-di-test) {
        case valore_1:
                istruzione_1;
        break;    //istruzione break opzionale
        case valore_2:
                istruzione_2;
                …
                istruzione_k;
        break;
        case valore_3:
        case valore_4:  { //blocco di codice opzionale
                istruzione_k+1;
                …
                istruzione_j;
                }
        break;
        default: { //clausola default opzionale
        istruzione_j+1;
        …
        istruzione_n;
        }
}
```

La parola chiave switch definisce il costrutto stesso ed è delimitato da un blocco di codice. Tra parentesi tonde dichiara una variabile o un'espressione (ovvero un'istruzione che ritorna un valore come potrebbe essere un metodo) che svolge lo stesso ruolo che svolgeva l'espressione booleana nel costrutto if. La differenza è che con l'espressione booleana dell'if si potevano definire espressioni come per esempio a<10 mentre per lo switch si tratto solo di un valore e non di un'espressione booleana.

Tale valore dovrebbe coincidere con uno dei valori specificati nella varie clausole case.

La parola chiave case viene seguita da un valore univoco costante che lo identifica (detta label, etichetta) e dopo i : vengono specificate le istruzioni da seguire se il valore di test coincide con il valore del case. In linea di massima il valore può essere un tipo intero primitivo (byte, short, char o int mentre long non è ammesso oppure classi Java che possono sostituire tali tipi primitivi, sono dette classi wrapper) o una classe di tipo String (in questo caso bisogna stare attenti nei controlli di uguaglianza in quanto esse sono case sensitive e possono contenere spazi e quindi sarà più facile introdurre errori).

Non è possibile dichiarare due case con la stessa label.

E' possibile non dichiarare un blocco di codice da eseguire per un etichetta andando ad accorpare più case per lo stesso blocco di codice (vedi case valore_3 e case valore_4).

La parola chiave default svolge il ruolo che l'else svolge nel costrutto if. E' l'equivalente di una clausola case che definisce le istruzione da eseguire se il valore del parametro di test dello switch non

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

coincide ad alcuna label dei case definiti. Le clausole case e default sono opzionali. La parola chiave break provoca l'immediata uscita dal costrutto, ed è anch'essa opzionale.

Vengono quindi eseguite le istruzioni che seguono la parola chiave case che definisce lo stesso valore che assume variabile-di-test.

Se il valore dei test non coincide con nessuno dei valori dei case dichiarati allora vengono eseguite le istruzioni che seguono la clausola default (se essa è presente, visto che è opzionale). Si consiglia comunque di usare la clausola default anche quando sembra non essercene bisogno. Infatti a priori non sappiamo se il programma evolverà ampliando i possibili valori che può assumere la variabile-di-test e quindi i possibili case . La clausola default potrebbe servire per gestire i nuovi case, con un comportamento standard, sia per scoprire che il costrutto switch deve essere modificato per accomodare i nuovi case.

E' anche consigliabile mantenere un ordine logico dei vari case (anche se l'ordine è in teoria indifferente) per non incorrere in dimenticanze e peggiorare la leggibilità.

A seconda del valore intero che assume la variabile di test vengono eseguite determinate espressioni. La variabile di test deve essere o di tipo stringa o di un tipo di dato compatibile con un intero, ovvero un byte, uno short, un char oppure direttamente un int (ma non un long).

Inoltre valore_1…valore_n devono essere espressioni costanti e diverse tra loro. Si noti che la parole chiave break provoca l'immediata uscita dal costrutto.

Il comando break fa saltare l'esecuzione alla parentesi graffa chiusa E.vita, ad esempio, che dopo aver eseguito il caso 3 venga eseguito anche il 4. Tra l'etichetta case N: e il comando break ci può essere più di un comando senza bisogno di parentesi graffe aggiuntive, ma è buona norma metterle.

Se dopo aver eseguito tutte le istruzioni che seguono un'istruzione di tipo case non è presente un'istruzione break, verranno eseguiti tutti gli statement (istruzioni) che seguono gli altri case, sino a quando non si arriverà ad un break (questa tecnica è nota come fall through).

Lo switch può essere usato per rappresenta quindi un modo alternativo all'if concatenato. In Fig. 5 l'esempio di Fig.6 rappresentato con uno switch.

```
…
switch ( giorno ) {
case 1: System.out.println (" Lunedi "); break;
case 2: System.out.println (" Martedi "); break;
case 3: System.out.println (" Mercoledi "); break;
case 4: System.out.println (" Giovedi "); break;
case 5: System.out.println (" Venerdi "); break;
case 6: System.out.println (" Sabato "); break;
case 7: System.out.println (" Domenica "); break;
default: System.out.println (" Numero errato ");
}
…
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Fig. 6: Esempio di switch con giorni della settimana

Il comando switch consente di accorpare casi in maniera abbastanza semplice come mostrato nell'esempio di Fig.7 sempre per la stampa dei giorni della settimana.

```
…
switch ( giorno ) {
case 1:
case 2:
case 3:
case 4:
case 5: System . out . println (" Giorno lavorativo "); break;
case 6: System . out . println (" Giorno prefestivo "); break;
case 7: System . out . println (" Giorno festivo "); break;
default: System . out . println (" Numero errato ");
}
…
```

Fig. 7: Esempio di switch con accorpamento di case

Se dopo aver eseguito tutte le istruzioni che seguono un'istruzione di tipo case non è presente un'istruzione break, verranno eseguiti tutti gli statement (istruzioni) che seguono gli altri case, sino a quando non si arriverà ad un break (questa tecnica è nota come fallthrough)

a. Consigli di utilizzo

E' sempre consigliabile usare la clausola default anche quando sembra non essercene bisogno. Infatti a priori non sappiamo se lo switch si evolverà ampliando il numero di case. La clausola default potrebbe servire sia per gestire i nuovi case, con un comportamento standard sia per scoprire che il costrutto switch deve essere modificato per accomodare i nuovi case. Di fatto la clausola default nel costrutto switch è l'equivalente della clausola else nel costrutto if

E' consigliabile mantenere un ordine logico dei vari case. Infatti, anche se è possibile ordinare le varie clausole in modo casuale senza alterare il funzionamento del costrutto (ad esempio spostando la clausola default in cima alla lista) con un ordine logico si riduce il rischio di incorrere in dimenticanze e si migliora la leggibilità.

E' consigliabile usare il break anche per la clausola default, pure se è posizionata come ultima clausola di un costrutto switch. Infatti, è possibile che in un aggiornamento del codice futuro (magari seguito da una persona diversa) si possa aggiungere un nuovo case, dopo la clausola default. Senza un break si eseguirebbero in sequenza le istruzioni del default seguite, non volute, da quelle del nuovo case.

E' consigliabile delimitare il pezzo di codice di ciascun case con delle parentesi graffe, sebbene non strettamente necessario. In tal modo le variabili locali all'interno di un case rimangano tali, altrimenti esse sarebbero visibili all'interno di tutti i case dello stesso costrutto dichiarati successivamente. Limitare la visibilità delle variabili il più possibile laddove sono dichiarate e usate è buona norme e usare un blocco di

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

codice (ovvero le istruzioni racchiuse da {} sebbene talvolta non strettamente necessario aiuta in tale direzione).

Abbiamo visto il costrutto switch nella sua definizione essenziale così come pensato all'inizio, tra l'altro come costrutto derivato da C e C++. Nelle varie versioni di Java tale costrutto è stato rivisto per essere usato come espressione (ovvero un'istruzione che ritorna un valore) e introducendo una sintassi meno verbosa andando anche ad evitare problemi in caso di un break dimenticato (dalla versione 13). Oggi switch è un costrutto nettamente più utile e potente rispetto alla versione originale, ma l'essenza di come funziona è quella descritta.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Errori tipici nell'utilizzo di if e switch

Come mostrato in Fig. 8, spesso capita di mettere il punto e virgola dopo l'espressione booleana dell'if perché abituati a farlo a fine riga. In realtà if-else è un unico costrutto al punto che non è necessario racchiuderlo tra le parentesi quando ad esempio viene annidato dentro un altro if.

Questo causa la stampa di «x vale 0» in ogni caso indipendentemente dal verificarsi della condizione, in quanto il costrutto if si considera finito con il ; (si tratta di un if vuoto di fatto).

Un altro errore tipico è non racchiudere tra {} le istruzioni multiple dentro il ramo di if e/o di else. Ne consegue che solo la prima istruzione viene eseguita in modo condizionato, le altre sono eseguite in ogni caso.

```
…
// if con comando vuoto
int x = 10;
if (x == 0); //il ; non ci vuole
System.out. println ("x vale 0"); //eseguita in ogni caso
…
// if-else senza blocchi (parentesi graffe)
if (x <= y) // manca {
System.out.print (x );
System.out.print ("e' minore o uguale di");
System.out.println (y );
// manca }
else // manca {
System.out.print (x );
System.out.print ("e' maggiore di");
System.out.println (y );
// manca }
```

Fig. 8: Esempi con errori tipici di uso di if

Nel caso dello switch come mostrato in Fig. 9 un errore comune è omettere il break che provoca l'esecuzione in sequenza delle istruzioni dal primo switch all'ultimo. Ricordiamo che il break è opzionale.

```
// omissione di break
switch ( giorno ) {
case 1: case 2: case 3: case 4: case 5:
       System.out.println (" Giorno lavorativo ");// manca break;
case 6:
       System.out.println (" Giorno prefestivo ");// manca break;
case 7:
       System.out.println (" Giorno festivo "); // manca break;
default:
       System.out.println (" Numero errato ");
}
```

Fig. 9: Esempi con errori tipici di uso di switch

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. Costrutti while e for

I cicli servono per poter iterare (ripetere) più volte delle operazioni, ad esempio, per eseguire una somma su un numero arbitrario di addendi forniti dall'utente. Il comando while consente di ripetere una istruzione (o un gruppo di istruzioni compresi in blocco di codice ovvero racchiuse in parentesi graffe) tante volte fintanto che una condizione specificata è vera. Ogni ciclo di esecuzione viene detto iterazione.

La sintassi è la seguente:

```
[inizializzazione;]
while (espressione_booleana) {
istruzione_1
…
istruzione_n;
[aggiornamento iterazione;]
}
```

Semantica del comando:

1. L'espressione booleana viene valutata
2. Se l'espressione booleana è vera si eseguono le istruzioni e si ricomincia dal punto 1
3. Se la condizione è falsa si salta il corpo e si procede con l'istruzione successiva al while

Un semplice esempio viene mostrato in Fig. 10.

```
// ciclo while
…
int i=1;
while (i<=10) {
System.out.println(i);
i++;
}
….
```

Fig. 10: Esempio con ciclo while

In primo luogo viene dichiarata ed inizializzata ad 1 una variabile intera i. Poi inizia il ciclo in cui è esaminato il valore booleano dell'espressione tra parentesi. Siccome i è uguale a 1, è minore di 10 e la condizione è verificata. Quindi viene eseguito il blocco di codice nel quale prima sarà stampato il valore di i (ovvero 1) e poi sarà incrementata la variabile stessa di un'unità. A questo punto, terminato il blocco di codice, verrà nuovamente testato il valore dell'espressione booleana con i che è diventato pari a 2. La

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

condizione è di nuovo verificata e quindi di nuovo verrà eseguito il blocco di codice con anche aggiornamento del valore di i. Questo si ripeterà fino a quando i verrà aggiornato a 11 e quindi nella successiva valutazione dell'espressione booleana, la condizione non si verifica e quindi il programma eseguirà l'istruzione successiva al while.

Si noti che all'interno del blocco di codice eseguito ci sono delle istruzioni che possono influire sul verificarsi o meno della cindizione al controllo successivo. In questo caso si ha un indice di iterazione i che viene inizializzato prima e poi incrementato dentro il ciclo così che ad un certo punto la condizione non viene più verificata (i diventa maggiore di 10).

Quando scriviamo un ciclo while, in generale potremmo non sapere quante iterazioni esso farà: dipende dal tasso di aggiornamento della variabile che viene valutata come parte dell'espressione booleana (decremento quantità di oggetti dalla cui esistenza dipendono le operazioni previste nel ciclo) oppure dal verificarsi di qualche condizione esterna di qualche tipo (arrivo di un messaggio su una coda).

In alcuni casi, invece, il numero di iterazioni è noto a priori Esempio: in SommaNumeri si chiede all'utente quanti numeri voglia sommare, e il numero inserito dall'utente diventa esattamente il numero di iterazioni del ciclo.

Quando il numero di iterazioni è noto a priori, in alternativa al while possiamo usare il comando for con la seguente sintassi:

```
for (inizializzazione; espressione booleana; aggiornamento) {
istruzione_1;
istruzione_2;
…
istruzione_n;
}
```

dove:

- inizializzazione è un comando eseguito all'inizio del ciclo
- espressione booleana per valutare se effettuare l'esecuzione delle istruzioni o uscire dal ciclo
- aggiornamento è un comando eseguito ad ogni iterazione

Le istruzioni possono essere n oppure anche una sola.

La sintassi è più compatta rispetto a quella del ciclo while. Le parentesi tonde di un ciclo for contengono parti separate da punti e virgola. La prima parte è dedicata alla inizializzazione delle variabili locali al ciclo che smetterà di esistere al termine del ciclo. Nella seconda parte invece, bisogna specificare la condizione che deve essere verificata affinché sia eseguita l'interazione. Infine la terza parte è dedicata ad

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

istruzioni (solitamente costituite dall'aggiornamento delle variabile inizializzata nella prima parte) che saranno eseguite automaticamente dopo ogni iterazione.

```
// ciclo for
…
for (int n=1; n<=10; n++) {
System.out.println(n);
}
….

for (int i=0, j=10; i<5 || j>5; i++, j--) {
System.out.println(i);
System.out.println(j);
}
…
```

Fig. 11: Esempi di uso del ciclo for

Nell'esempio in Fig. 11 dichiariamo una varabile locale n che smetterà di esistere al termine del ciclo. Nella seconda parte invece, è specificata la condizione che ovvero che n deve essere minore o uguale di 10. Infine la terza parte le istruzioni che saranno eseguite automaticamente dopo ogni iterazione, in questo caso l'incremento di n che avverrà dopo ogni iterazione.

In questo esempio si stamperanno i numeri da 1 a 10 perché il primo aggiornamento avverrà solo dopo la prima iterazione e si esce con n=11 dopo aver stampato 10 e incrementato la variabile di una unità. In sostanza vengono stampati tutti i numeri fino a 10 compreso.

E' buona norma non modificare i nel corpo del for .

Si possono anche dichiarare più variabili all'interno del ciclo for, più aggiornamenti e, sfruttando operatori condizionali, anche più condizioni. Vedi il secondo esempio.

Come si vede le dichiarazioni vanno separate da virgole e hanno il vincolo di dover essere tutte dello stesso tipo (in questo caso int). Anche gli aggiornamenti vanno separati da virgole ma in questo caso non ci sono vincoli. L'espressione booleana è in questo caso legata ad una condizione logica sul valore di entrambe.

I nomi i,j,k... sono usati comunemente per le variabili-contatore dei cicli for .

b. Consigli di utilizzo

Il ciclo for è probabilmente il ciclo più utilizzato vista la sua grande versatilità. Inoltre è l'unico ciclo che permette di dichiarare una variabile contatore con visibilità interna al ciclo stesso e quindi si può riutilizzare in un for successivo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quando si esegue un ciclo sfruttando un indice (ad esempio su un array i cui elementi sono caratterizzati da un indice) il ciclo for è indubbiamente il più adatto. Per esempio riprendendo l'array alfabeto potremmo stampare tutti i suoi elementi in modo compatto e al tempo stesso chiaro, come mostrato nell'esempio di Fig. 12:

```
// ciclo for sugli array
…
char alfabeto [] =
{'a','b','c','d','e','f','g','h','i','l','m','n','o'
'p','q','r','s','t','u','v','z'};
for (int i=0; i<21; i++) {
System.out.println (alfabeto[i]);
}
….
```

Fig. 12: Esempio di uso del ciclo for sugli array

Il ciclo while è utilizzato soprattutto quando non si può calcolare a priori il numero di iterazioni da effettuare e non si ha un indice da aggiornare ad ogni iterazione. Ad esempio nei cicli infiniti dove la sintassi è molto semplice.

```
// uso ottimale ciclo while
…
while (true) { … }
for (;true;){…}  // equivalente a for(;;){…}
….
public class Interruttore {
public static final int ACCESO=0;
public static final int SPENTO=1;
public int posizione;

public void accendiLampadina () {
       while (interruttore.posizione == ACCESO) {
       //codice omesso
       }
}
}
```

Fig. 13: Esempi di uso del ciclo while

Più in generale il ciclo while è adatto a dichiarare espressioni booleane con una logica basata sullo stato (state based logic). In particolare si dice che lo stato di un oggetto è definito dal valore delle sue variabili di istanza. Un oggetto quindi a runtime può assumere vari stati a seconda del cambiamento del

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

valore delle sue variabili di istanza. Il ciclo while ben si adatta a gestire gli stati di un oggetto. Come ad esempio possiamo considerare il ciclo while interno al metodo accendiLampadina della classe Interruttore che segue mostrato in Fig.13 dove ACCESO/SPENTO sono in sostanza due costanti che valgono per ogni istanza della classe. posizione invece è la situazione corrente dell'interruttore ovvero se è chiuso (ACCESO) oppure aperto (SPENTO). Verrà modificato in qualche parte del programma in cui la classe è usata. Nel ciclo while si mette del codice che verrà eseguito finché l'interruttore è in posizione di ACCESO per tenere la lampadina illuminata.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

7. Costrutti do-while e foreach

Il ciclo do-while è il ciclo meno utilizzato. E' però fondamentale nei casi in cui si desideri che le istruzioni del blocco di codice siano eseguite almeno una volta. La sintassi è la seguente:

```
[inizializzazione;]
do {
istruzione_1
…
istruzione_n;
[aggiornamento iterazione;]
} while (espressione_booleana);
```

Viene prima eseguito il blocco di codice e poi viene valutata l'espressione booleana (condizione di terminazione) che si trova a destra della parola chiave while. Se l'espressione booleana è verificata viene rieseguito il blocco di codice, altrimenti il ciclo termina.

Per esempio consideriamo questo semplice frammento di codice:

```
int i=10;
do {
System.out.println(i);
} while (i<10);
```

L'output è semplicemente singolo con la stampa di 10 a video. La prima iterazione è eseguita comunque anche se la condizione poi si rivela falsa.

Dalla versione 5 di Java è stata aggiunta una nuova versione del ciclo for con sintassi molto semplice e compatta:

```
for (variabile_temporanea : oggetto_iterabile) {
istruzione_1;
istruzione_2;
…
istruzione_n;
}
```

Dove oggetto_iterabile è l'array sui cui elementi si vuole iterare, variabile_termporanea dichiara una variabile alla quale, durante l'esecuzione del ciclo, sarà assegnato il valore dell'i-esimo elemento

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

dell'oggetto_iterabile all'i-esima iterazione. Quindi variabile_temporanea rappresenta all'interno del blocco di codice di questo ciclo un elemento dell'oggetto_iterabile. Solitamente le istruzione del ciclo utilizzano tale variabile. A questo punto non è applicabile un'espressione booleana per la quale il ciclo deve terminare; questo è già indicativo del fatto che questo for viene usato soprattutto quando si vuole iterare su tutti gli elemento di un oggetto iterabile. In Fig. 14 è mostrato un esempio che stampa a video tutti gli elementi dell'array.:

```
// ciclo foreach
…
int[] arr = {1,2,3,4,5,6,7,8,9};
for (int tmp : arr) {
system.out.println (tmp);
}
//stampa a video tutti gli elementi dell'array.
```

Fig. 14: Esempio di uso del ciclo for migliorato

Questa versione di for ha diversi limiti rispetto al ciclo for tradizionale. Per esempio non è possibile eseguire cicli all'indietro e non è nemmeno possibile farlo su più oggetti contemporaneamente. Infine non è possibile accedere all'indice dell'array dell'elemento corrente, ma per ovviare a questo si può dichiarare un contatore all'esterno del ciclo e incrementarlo all'interno. In casi come questo sarebbe opportuno utilizzare un semplice ciclo while.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- [1] Il nuovo Java, Claudio De Sio Cesari, Hoepli Informatica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).